

Simple Command Sequencer Generator

Autocoder User Guide

1 Description

The purpose of this generator is to provide a user-friendly way of defining named command sequences for the Simple Command Sequencer component. The generator takes a YAML model file as input which declares a set of sequences – each a named, ordered list of sub-commands (and optional sleeps) – and autocodes both the Ada specification of the sequence step tables consumed by the sequencer’s Init routine *and* a set of first-class per-sequence wrapper commands so that each sequence appears on the sequencer instance as its own typed, operator-callable command.

Note the example shown in this documentation is used in the unit test of this component so that the reader of this document can see it being used in context. Please refer to the unit test code for more details on how this generator can be used.

2 Schema

The following pykwalify schema is used to validate the input YAML model. Model files must be named in the form *optional_name.assembly_name.command_sequences.yaml*, where *optional_name* is the specific name of this set of sequences (only needed if more than one suite is defined in the assembly) and *assembly_name* is the assembly the sequences will be used in. Generally this file is created in the same directory or near to the assembly model file. The schema is commented to show each available YAML key and what it accomplishes; even without knowing the specifics of pykwalify schemas, you should be able to glean the available structure from the file below.

```
1  # Schema for command_sequences YAML files
2  # This defines the structure and validation rules
3
4  type: map
5  mapping:
6    name:
7      required: true
8      type: str
9      desc: |
10         The name of the command sequences package. This will be used as the
11         Ada package name, so it must follow Ada naming conventions.
12
13    description:
14      required: false
15      type: str
16      desc: |
17         Description of this command sequences package. This will appear in
18         the generated Ada code comments.
19
20    preamble:
21      required: false
22      type: str
23      desc: |
24         Custom preamble text to include in the generated Ada package.
```

```

25
26 command_timeout_seconds:
27     required: false
28     type: int
29     range: { min: 0 }
30     desc: |
31         Default timeout, in whole seconds, that the sequencer will wait for
32         a sub-command's response before declaring the sequence failed. Used
33         for any sequence in this suite that does not specify its own
34         command_timeout_seconds.
35
36 with:
37     required: false
38     type: seq
39     desc: |
40         List of additional Ada packages to "with" in the generated sequences
41         package. Use this when a step's arg expression references a package
42         that is not pulled in automatically, e.g. an enum package for a
43         qualified enumeration literal.
44     sequence:
45         - type: str
46
47 sequences:
48     required: true
49     type: seq
50     desc: |
51         List of command sequence definitions. Each sequence defines a named
52         command that executes a series of sub-commands.
53     sequence:
54         - type: map
55           mapping:
56             name:
57                 required: true
58                 type: str
59                 desc: |
60                     Name of this sequence. This will become a command that can be
61                     executed. Must follow Ada naming conventions.
62
63             description:
64                 required: false
65                 type: str
66                 desc: |
67                     Description of what this sequence does. Will appear in generated
68                     code and documentation.
69
70             arg_type:
71                 required: false
72                 type: str
73                 desc: |
74                     The Ada type for this sequence's argument. If not specified,
75                     the sequence takes no arguments. Must be a valid Ada type name,
76                     possibly qualified (e.g., "Power_Mode.T" or "Packed_U16.T").
77
78             wait_for_command_completion:
79                 required: false
80                 type: bool
81                 default: true
82                 desc: |
83                     Default setting for whether to wait for each sub-command to
84                     complete before executing the next one. Individual steps can
85                     override this.

```

```

86
87     continue_on_failure:
88         required: false
89         type: bool
90         default: false
91         desc: |
92             If true, the sequence will continue executing even if a
93             ↪ sub-command
94             fails. If false, the sequence will abort on the first failure.
95
96     command_timeout_seconds:
97         required: false
98         type: int
99         range: { min: 0 }
100        desc: |
101            Override the suite-level default timeout for sub-command
102            responses, in whole seconds, for this sequence only.
103
104    sequence:
105        required: true
106        type: seq
107        desc: |
108            The ordered list of steps to execute. Each step is either a
109            command dispatch (`command:` + optional `arg:`) or an in-sequence
110            sleep (`sleep:` + Packed_U32 milliseconds expression). The two
111            forms are mutually exclusive; exactly one must be present per
112            step. Validation is enforced in the model loader.
113        sequence:
114            - type: map
115              mapping:
116                command:
117                    required: false
118                    type: str
119                    desc: |
120                        The command to execute in the format
121                        ↪ "Component.Command_Name".
122                        The component must be a valid component instance name in
123                        ↪ your
124                        assembly, and the command must exist on that component.
125                        Mutually exclusive with `sleep:`.
126
127                arg:
128                    required: false
129                    type: str
130                    desc: |
131                        An Ada expression that evaluates to the argument for this
132                        command. You can reference the parent sequence's argument
133                        using "Arg.field_name" syntax. If the command takes no
134                        argument, omit this field. Only valid alongside
135                        ↪ `command:`.
136
137                Examples:
138                    - "Arg" - pass through the entire sequence argument
139                    - "Arg.Power_Mode" - pass a field from the sequence
140                      ↪ argument
141                    - "(Mode => Arg.Mode, Enable => True)" - construct a
142                      ↪ record
143                    - "42" - a literal value
144
145        wait_for_completion:
146            required: false

```

```

141         type: bool
142         desc: |
143             Whether to wait for this specific command to complete
144             ↪ before
145             proceeding. If not specified, uses the parent sequence's
146             wait_for_command_completion setting. Only valid alongside
147             `command:`.
148
149     sleep:
150         required: false
151         type: str
152         desc: |
153             An Ada expression that evaluates to a Packed_U32.T value
154             representing milliseconds to pause the sequence. Not
155             bus-routed; the sequencer frame parks itself for the
156             duration without issuing any command. Mutually exclusive
157             with `command:`/`arg:`/`wait_for_completion:`.
158
159         Examples:
160             - "(Value => 3000)" - sleep 3 seconds
161             - "(Value => Arg.Pause_Millis)" - sleep duration from
162               ↪ sequence arg
163
164 # Additional validation rules (enforced in the model Python code):
165 # - Command format must be "Component.Command" with valid identifiers, or
166 #   the reserved name "Sleep"
167 # - Referenced types in arg_type must exist in the system
168 # - Argument expressions must be valid Ada syntax
169 # - Component and command names must match instances in the assembly

```

3 Example Input

The following is an example command sequences input YAML file. It defines five sequences exercising the main features: an arg-less sequence with both statically-armed commands and a sleep, a sequence with a typed argument record whose fields are dispatched as dynamic arguments to two sub-commands, a no-wait sequence used for direct-dispatch testing, a sequence with a single large sleep that exercises the out-of-range protection, and a sequence that calls another sequence as a sub-sequence step. This example adheres to the schema shown in the previous section, and is commented to give clarification.

Note the optional top-level `with:` key, which lists additional Ada packages to “with” in the generated package. A step’s `arg:` expression is emitted verbatim into the generated Ada, so any package it references that is not pulled in automatically – such as an enum package needed to qualify an enumeration literal – must be listed here.

```

1  # Command Sequences Definition
2  # This file defines composite commands that execute sequences of sub-commands
3
4  name: Flight_Sequences
5  description: Common flight software command sequences
6
7  command_timeout_seconds: 10 # default wait for sub-command responses
8
9  # Extra packages needed by step arg expressions below.
10 with:
11     - Sequence_Enums
12
13 sequences:

```

```

14  # Simple sequence with no arguments
15  - name: Sequence_A
16    description: Execute commands on component A
17    wait_for_command_completion: true
18    continue_on_failure: true
19
20    sequence:
21      - command: Component_A.Command_1
22      - command: Component_A.Command_2
23        arg: (Seconds => 3, Subseconds => 14)
24      - sleep: (Value => 3000)
25      - command: Component_A.Command_3
26        arg: (Value => 3)
27
28  - name: Sequence_B
29    description: Perform generic arguments with component A and B
30    wait_for_command_completion: true
31    continue_on_failure: false
32    arg_type: Sequence_B_Arg.T
33
34    sequence:
35      - command: Component_A.Command_3
36        arg: Arg.Component_A_Arg
37      - command: Component_B.Command_3
38        arg: Arg.Component_B_Arg
39
40  - name: Sequence_C
41    description: No-wait sequence for direct-dispatch testing
42    wait_for_command_completion: false
43    continue_on_failure: false
44
45    sequence:
46      - command: Component_A.Command_1
47      - command: Component_A.Command_1
48
49  - name: Sequence_D
50    description: Sequence with a large sleep value to test an overflow sleep
51    wait_for_command_completion: true
52    continue_on_failure: false
53
54    sequence:
55      - sleep: (Value => 4_294_967_295)
56
57  # A sequence that calls another sequence. Each sequence is a first-class
58  # command on the sequencer, so a sub-sequence step is just a command step
59  # targeting the sequencer's own synthesized command.
60  - name: Sequence_E
61    description: Call Sequence_C as a sub-sequence, then run a regular command
62    wait_for_command_completion: true
63    continue_on_failure: false
64
65    sequence:
66      - command: Sequencer.Sequence_C
67        arg: (Response_Behavior =>
68          ↳ Sequence_Enums.Sequence_Response_Behavior.Send_After_Sequence_Start)
69      - command: Component_A.Command_1

```

4 Example Output

The example input above produces the following Ada outputs. The package spec declares the per-dynamic-step Resolver types, a step array constant per sequence, and the Sequences_Table constant (plus a stable Sequences access pointer). The package body implements the Resolver Resolve functions, which deserialize the sequence's per-call argument buffer through the input type's `Serialization.From_Byte_Array` and re-serialize the addressed field as the sub-command's argument buffer. The body is only emitted when the suite contains at least one dynamic step – a suite of purely static or sleep-only sequences declares no Resolve subprograms and so needs no body package.

The Sequences access should be passed into the Simple Command Sequencer's `Init` procedure during assembly initialization, e.g. in the assembly YAML:

```
- type: Simple_Command_Sequencer
  name: My_Sequencer
  init:
    - "Num_Concurrent_Sequences => 4"
    - "Sequences => Test_Assembly_Command_Sequences_Example_Sequences.Sequences"
```

```
1  -- Auto-generated by Adamant command sequences generator.
2  -- Do not edit by hand.
3  -- Common flight software command sequences
4  with Simple_Sequencer_Types; use Simple_Sequencer_Types;
5  with Test_Assembly_Commands; use Test_Assembly_Commands;
6  with Scs_Arg_Resolver;
7  with Command_Types;
8  with Basic_Types;
9  with Sequence_Arg_Utills; use Sequence_Arg_Utills;
10 with Sequence_Enums;
11 with Sys_Time_32;
12 with Packed_U32;
13 with Sequence_B_Arg;
14 with Sequence_C_Run_Arg;
15 package Test_Assembly_Command_Sequences_Example_Sequences is
16
17     -- Resolver types - one per dynamic step, each encodes a traversal path.
18     -----
19
20     type Sequence_B_Step_0_Resolver_T is new Scs_Arg_Resolver.T with null record;
21
22     overriding function Resolve (
23         Resolver : Sequence_B_Step_0_Resolver_T;
24         Data      : Basic_Types.Byte_Array
25     ) return Command_Types.Command_Arg_Buffer_Type;
26
27     Sequence_B_Step_0_Resolver : aliased constant
28         Sequence_B_Step_0_Resolver_T := (null record);
29
30
31     type Sequence_B_Step_1_Resolver_T is new Scs_Arg_Resolver.T with null record;
32
33     overriding function Resolve (
34         Resolver : Sequence_B_Step_1_Resolver_T;
35         Data      : Basic_Types.Byte_Array
36     ) return Command_Types.Command_Arg_Buffer_Type;
37
38     Sequence_B_Step_1_Resolver : aliased constant
39         Sequence_B_Step_1_Resolver_T := (null record);
40
41     -----
```

```

42  -- Step arrays - one constant per sequence.
43  -----
44  Sequence_A_Steps : aliased constant Step_Array :=
45  [
46      0 => (Kind          => Command_Step,
47             Id           => Component_A_Command_1,
48             Arg_Length   => 0,
49             Arg          => [others => 0]),
50      1 => (Kind          => Command_Step,
51             Id           => Component_A_Command_2,
52             Arg_Length   =>
53             ↪ Sys_Time_32.Serialization.Serialized_Length,
54             Arg          => To_Arg
55             ↪ (Sys_Time_32.Serialization.To_Byte_Array
56             ↪ ((Seconds => 3, Subseconds => 14)))),
57      2 => (Kind          => Sleep,
58             Id           => 0,
59             Arg_Length   => 0,
60             Sleep_Arg    => (Value => 3000)),
61      3 => (Kind          => Command_Step,
62             Id           => Component_A_Command_3,
63             Arg_Length   =>
64             ↪ Packed_U32.Serialization.Serialized_Length,
65             Arg          => To_Arg
66             ↪ (Packed_U32.Serialization.To_Byte_Array ((Value
67             ↪ => 3))))
68  ];
69  Sequence_B_Steps : aliased constant Step_Array :=
70  [
71      0 => (Kind          => Runtime_Argument_Command_Step,
72             Id           => Component_A_Command_3,
73             Arg_Length   =>
74             ↪ Packed_U32.Serialization.Serialized_Length,
75             Resolver     => Scs_Arg_Resolver.T_Access'(Sequence_B_St
76             ↪ ep_0_Resolver'Access)),
77      1 => (Kind          => Runtime_Argument_Command_Step,
78             Id           => Component_B_Command_3,
79             Arg_Length   =>
80             ↪ Packed_U32.Serialization.Serialized_Length,
81             Resolver     => Scs_Arg_Resolver.T_Access'(Sequence_B_St
82             ↪ ep_1_Resolver'Access))
83  ];
84  Sequence_C_Steps : aliased constant Step_Array :=
85  [
86      0 => (Kind          => Command_Step,
87             Id           => Component_A_Command_1,
88             Arg_Length   => 0,
89             Arg          => [others => 0]),
90      1 => (Kind          => Command_Step,
91             Id           => Component_A_Command_1,
92             Arg_Length   => 0,
93             Arg          => [others => 0])
94  ];
95  Sequence_D_Steps : aliased constant Step_Array :=
96  [
97      0 => (Kind          => Sleep,
98             Id           => 0,
99             Arg_Length   => 0,
100            Sleep_Arg    => (Value => 4_294_967_295))
101  ];
102  Sequence_E_Steps : aliased constant Step_Array :=

```

```

93     [
94         0 => (Kind          => Command_Step,
95              Id            => Sequencer_Sequence_C,
96              Arg_Length => Sequence_C_Run_Arg.Serialization.Seriali
97                  ↳ lized_Length,
98              Arg          => To_Arg
99                  ↳ (Sequence_C_Run_Arg.Serialization.To_Byte_Array
100                  ↳ (((Response_Behavior => Sequence_Enums.Sequence_)
101                  ↳ Response_Behavior.Send_After_Sequence_Start))))),
102         1 => (Kind          => Command_Step,
103              Id            => Component_A_Command_1,
104              Arg_Length => 0,
105              Arg          => [others => 0])
106     ];
107
108 -----
109 -- Sequences_Table
110 -----
111 Sequences_Table : aliased constant Sequences_Type :=
112 [
113     0 => (
114         Wait_For_Cmd_Resp      => True,
115         Abort_On_Failed_Cmd    => False,
116         Command_Timeout_Millis => 10000,
117         Steps                  => Sequence_A_Steps'Access),
118     1 => (
119         Wait_For_Cmd_Resp      => True,
120         Abort_On_Failed_Cmd    => True,
121         Command_Timeout_Millis => 10000,
122         Steps                  => Sequence_B_Steps'Access),
123     2 => (
124         Wait_For_Cmd_Resp      => False,
125         Abort_On_Failed_Cmd    => True,
126         Command_Timeout_Millis => 10000,
127         Steps                  => Sequence_C_Steps'Access),
128     3 => (
129         Wait_For_Cmd_Resp      => True,
130         Abort_On_Failed_Cmd    => True,
131         Command_Timeout_Millis => 10000,
132         Steps                  => Sequence_D_Steps'Access),
133     4 => (
134         Wait_For_Cmd_Resp      => True,
135         Abort_On_Failed_Cmd    => True,
136         Command_Timeout_Millis => 10000,
137         Steps                  => Sequence_E_Steps'Access)
138 ];
139
140 Sequences : constant Sequences_Access := Sequences_Table'Access;
141
142 end Test_Assembly_Command_Sequences_Example_Sequences;

```

```

1  -- Auto-generated by Adamant command sequences generator.
2  -- Do not edit by hand.
3  with Sequence_B_Arg;
4  with Packed_U32;
5  package body Test_Assembly_Command_Sequences_Example_Sequences is
6
7      overriding function Resolve (
8          Resolver : Sequence_B_Step_0_Resolver_T;
9          Data      : Basic_Types.Byte_Array
10      ) return Command_Types.Command_Arg_Buffer_Type is

```



```

11      -- Deserialize the sequence's per-call arg buffer through the input
12      -- type's safe deserializer instead of reinterpreting raw bytes via
13      -- 'Address. The slice is sized to the input type's serialized length.
14      Input : constant Sequence_B_Arg.T :=
15          Sequence_B_Arg.Serialization.From_Byte_Array (
16              Data (Data'First .. Data'First +
17                  ↪ Sequence_B_Arg.Serialization.Serialized_Length - 1));
18  begin
19      return To_Arg (
20          Packed_U32.Serialization.To_Byte_Array (
21              Input.Component_A_Arg
22          )
23      );
24  end Resolve;
25
26  overriding function Resolve (
27      Resolver : Sequence_B_Step_1_Resolver_T;
28      Data      : Basic_Types.Byte_Array
29  ) return Command_Types.Command_Arg_Buffer_Type is
30      -- Deserialize the sequence's per-call arg buffer through the input
31      -- type's safe deserializer instead of reinterpreting raw bytes via
32      -- 'Address. The slice is sized to the input type's serialized length.
33      Input : constant Sequence_B_Arg.T :=
34          Sequence_B_Arg.Serialization.From_Byte_Array (
35              Data (Data'First .. Data'First +
36                  ↪ Sequence_B_Arg.Serialization.Serialized_Length - 1));
37  begin
38      return To_Arg (
39          Packed_U32.Serialization.To_Byte_Array (
40              Input.Component_B_Arg
41          )
42      );
43  end Resolve;
44
45  end Test_Assembly_Command_Sequences_Example_Sequences;

```

5 Per-Sequence First-Class Commands

In addition to the sequence tables above, this same model drives the autocoded synthesis of one first-class wrapper command per declared sequence on the Simple Command Sequencer instance. Discovery is build-path-driven: the sequencer's commands model queries the model database for every `*.command_sequences.yaml` in scope, so a naked component build (no assembly) sees no sequences and produces only the three static commands (`Run_Sequence`, `Kill_All_Sequences`, `Set_Summary_Packet_Period`), while an assembly build whose path includes a sequences suite adds one wrapper command per sequence. Each wrapper command (e.g. `Sequence_B`) takes the sequence's wrapped argument record (the sequence's native arg type plus a trailing `Response__Behavior` flag) and, at run time, packs that typed argument into a generic `Run_Sequence__Arg.T` and dispatches through the existing `Run_Sequence` backbone using the sequence's runtime slot index. The wrapped argument records are produced from this same model by the companion `wrapped_run_args` generator, and the handler bodies for the synthesized commands live in a generated intermediate that the component's implementation derives from – no per-sequence Ada needs to be hand-written.

6 Sequences Calling Sequences

Because every sequence is a first-class command on the sequencer instance, a sequence step may invoke another sequence simply by targeting the sequencer's own synthesized command, e.g. `command:`

`My_Sequencer.Sequence_C`. No special syntax or generator support is involved – the step resolves like any other `Component.Command` reference, provided the sequencer instance is declared in the assembly. At run time the sub-sequence command is emitted out the sequencer’s `Command_T_Send` connector, routed through the command router, and arrives back at the sequencer’s `Command_T_Recv_Async`, where it claims its own frame and executes concurrently with (and independently of) the calling sequence. If the calling sequence waits for command completion, the sub-sequence’s command response (per its `Response_Behavior` argument, see below) is what wakes it. Nesting depth is naturally capped by `Num_Concurrent_Sequences`: a sub-sequence call when no frame is available is rejected with a `No_Frame_Available` event and a `Failure` response to the calling frame.

7 Sequence Response Behavior

Every synthesized per-sequence command (and the generic `Run_Sequence` command) carries a `Response_Behavior` enumeration argument controlling when the sequencer replies to the command that started the sequence:

- `Send_After_Sequence_Start` (the conventional default) – the command response is sent immediately once the sequence has been accepted and started on a frame. This is the right choice for fire-and-forget operation and for sub-sequence calls where the caller should proceed as soon as the sub-sequence is running.
- `Send_After_Sequence_Completion` – the command response is withheld until the sequence ends, and its status reflects the outcome: `Success` when the final step completes, or `Failure` when the sequence is aborted by a failed sub-command, times out waiting for a sub-command response, encounters an out-of-range sleep or timeout, or is halted by `Kill_All_Sequences`. The sequencer captures the originating command’s source and id per frame, so concurrent deferred sequences each reply to their own operator.

A sequence step may forward a field of the parent sequence’s argument to the sub-command it dispatches, using the `Arg.field.subfield` syntax in the step’s `arg:`. Each such reference produces a typed `Resolver` in the generated package: at run time the sequencer dispatches that `Resolver`, which deserializes the per-call argument buffer through the input type’s safe `Serialization.From_Byte_Array`, navigates to the addressed field, and serializes it as the sub-command’s argument buffer. The resolver’s signature accepts only a `Basic_Types.Byte_Array` – there is no ‘Address reinterpretation’ – so the dynamic-argument machinery is type-safe by construction; a resolver cannot overlay an arbitrary record onto raw memory, only deserialize a byte array through the input type’s generated deserializer.

8 Sleep Steps

A sequence step may be a sleep instead of a command: `sleep: (Value => 3000)` pauses the executing frame for 3 seconds without issuing any bus traffic. Sleep is not a bus command; it is interpreted intrinsically by the sequencer frame state machine. The sleep expression is a `Packed_U32.T` aggregate in milliseconds and may itself be dynamic, e.g. `sleep: (Value => Arg.Pause_Millis)`, in which case the duration is taken from the sequence’s per-call argument. Out-of-range sleep durations (those that cannot be represented as a `Time_Span` on the platform, or that would overflow `Sys_Time` arithmetic when added to the current time) are rejected at run time with a `Sequence_Out_Of_Range_Sleep` event so the operator can see that the step was skipped.